OpenShift Cert Manager

Cert-manager

- Cert-manager
  - What is cert-manager
  - Why cert-manager?
  - How its installed
  - How it works
    - Certificates
    - Keyfactor Required Metadata
    - Approver-policy
      - All Environments
    - Issuers
    - Use case Object Reference Examples
      - Certificate.yaml example
      - Ingress.yaml example
  - Upgrade/Patching schedule
    - Validation
  - How do I validate the cert I created via cert-manager
  - What do i have to do as an app owner to migrate to keyfactor
  - Redis Certificate Limitations
- FYI
  - Route terminations
  - References:

Overview	
Scope	OpenShift Container Platform 4
Status	IN PROD
Audience	Technology Operations - Containers DevSecOps Cloud C4E Engineering
Author	Mulder, Rick
Maintainer	IPS Containers
Last Updated	09 Sep 2025
Table of Contents	• What is cert-manager • Why cert-manager? • How its installed • How it works ◦ Certificates ◦ Keyfactor Required Metadata ◦ Approver-policy ▪ All Environments ◦ Issuers ◦ Use case Object Reference Examples ▪ Certificate.yaml example ▪ Ingress.yaml example • Upgrade/Patching schedule ◦ Validation • How do I validate the cert I created via cert-manager • What do i have to do as an app owner to migrate to keyfactor • Redis Certificate Limitations • Route terminations • References:

What is cert-manager

Cert-manager adds certificates and certificate issuers as resource types in Kubernetes clusters. It simplifies the process of obtaining, renewing and using those certificates.

It can issue certificates from a variety of supported sources, including Let's Encrypt, HashiCorp Vault, and Venafi as well as private PKI. IN SVB we will be using the privake PKI internal to SVB.

It will ensure certificates are valid and up to date, and attempt to renew certificates at a configured time before expiry.

blocked URL

Why cert-manager?

Currently certificate renewals on Openshift Container Platform (OCP) are only semi-automated. Cert expiry tracking and renewal requests are manual process and are error prone.

Cert-manager simplifies the process of obtaining, renewing and using certificates. It helps in automated issuance and renewal of TLS certificates. It supports certificates from a variety of supported sources, including Venafi and private PKI. It also helps in securing pod to pod communication with mTLS using private PKI Issuers.

How its installed

In SVB cert-manager consist of 3 components: cert-manager, cert-manager-approver-policy, and command-issuer (Keyfactor integration). The installation involves 2 steps

- The cert-manager objects will be installed via the OpenShift GitOps Operator (Argo) and is dependent on our Keyfactor integration to issue and sign Certificates.
- Cert Manager accepts various issuer types. We previously supported SVB PKI, and moving forward will only support Keyfactor to align with the bank-wide PKI strategy.
- Once approved internally by our policies, the Keyfactor command component submits the certificate request to Keyfactor where it will make a final decision

How it works

The only supported method for requesting certificates through cert-manager/Keyfactor is by creating/requesting Certificate resource. This can be done by defining an object of "kind: Certificate" similar to deployments/sercices/routes.. Lets first look at the 3 main components that are involved in the process before diving into how it works

Certificates

The `Certificate` resource represents a human readable definition of a certificate request that is to be created for the objects/endpoints that needs a certificate to be created. This can be done by defining the certificate.yaml in the corresponding project with the required parameters. A `Certificate` resource specifies fields that are used to generate certificate signing requests which are then fulfilled by the issuer type you have referenced.

Example of a certificate object is below

Certificate Object

```
        apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: <name-of-the-cert> (eg: aqua-gateway)
  namespace: <your-namepsace> (eg: aqua)
  annotations:
    metadata.command-issuer.keyfactor.com/Certificate-Support-Group-
Contact: my-email-distro@firstcitizens.com
    metadata.command-issuer.keyfactor.com/Environment: Non-Prod-UAT
    metadata.command-issuer.keyfactor.com/Description: "AquaSec Gateway
Component Certificate"
    metadata.command-issuer.keyfactor.com/Organization: FCB
    metadata.command-issuer.keyfactor.com/Application-ID: APM00012345
    metadata.command-issuer.keyfactor.com/Platform: OPENSHIFT
spec:
  commonName: <your-dns>.ocp.dv-p.ocp.fcbint.net (eg: aqua-gateway.apps.dv-
p.ocp.fcbint.net)
  dnsNames:
  - <your-dns>.dv-p.ocp.fcbint.net (eg: aqua-gateway.apps.dv-p.ocp.fcbint.
net)
  - <your-dns>.dv-s.ocp.fcbint.net (eg: aqua-gateway.apps.dv-s.ocp.fcbint.
net)
  duration: 2160h0m0s
  issuerRef: ## SEE ISSUERS BELOW
    group: command-issuer.keyfactor.com
    kind: ClusterIssuer
    name: keyfactor
  privateKey:
    algorithm: RSA
    size: 4096
    rotationPolicy: Always
  secretName: <secretname-to-store-your-certificate> (eg: aqua-gateway-tls)
  secretTemplate:
    labels:
      app.kubernetes.io/name:
      app.kubernetes.io/instance:
      app.kubernetes.io/component:
      fcbint.net/app-id: ""
```

Keyfactor Required Metadata

> Failure to provide this metadata on your certificate will cause Keyfactor to reject your request.

Keyfactor requires that each certificate provides the following annotations/metadata:

- metadata.command-issuer.keyfactor.com/Certificate-Support-Group-Contact:
  - Your team's email distribution. You will receive email notices from Keyfactor regarding your certificate at this address.
  - This address is required to be either svb.com or firstcitizens.com email.
- metadata.command-issuer.keyfactor.com/Environment:
  - Supported values:
    - Non-Prod-DEV
    - Non-Prod-QA
    - Non-Prod-UAT # Recommended value for all LSVB Preprod
    - Prod-Corp
- metadata.command-issuer.keyfactor.com/Description:

    - This *must* be a non-nil string. We recommend a short description of the service.
- metadata.command-issuer.keyfactor.com/Organization:
    - The *only* supported value is: `FCB`
- metadata.command-issuer.keyfactor.com/Application-ID:
    - ServiceNOW Application ID of your application as displayed in the Business application table in CMDB, e.g. `APM00012345` this is added to all App namespaces for reference for those who dont have access to SNOW to look it up
- metadata.command-issuer.keyfactor.com/Platform:
    - The *only* supported value is: `OPENSHIFT`

Approver-policy

- This is the set of definitions that enables one to write the rules/policies to automatically manage the certificate issuance. One can control the issuance or denial of certificates by defining more than 1 approver policy.
- When a CertificateRequest is created it will be evaluated against all the approver-policies that are defined. If the certificate request abides by those rules the certificate request is approved other wise they are denied.
- CertificateRequestPolicy are split into 4 parts; `allowed`, `constraints`, `selector`, and `plugins`.
    - **Allowed** are the attributes that has be to explicitly defined in the request. If an allowed field is omitted, that attribute is considered "deny all" for requests.
        - Under allowed we can also add a flag "required = true" to force an attribute to be present. If this attribute is absent then the request will be denied. In SVB we have the commonName attribute set as true. So if this field is not defined then the CRs will be rejected.
- **Constraints:** Constraints is the block that is used to limit what attributes the request can have. If a constraint is not defined, then the attribute is considered "allow all".
    - In SVB MaxDuration and privateKey Algorithm and KeySize are constrains. Their values for various approver policies are defined below.

NOTE: The CommonName is a REQUIRED field in your certificate object. Try using the same syntax as below and fill in your App specific details

All Environments

Approval Policy name	Allowed DNS	CommonName (required field)

`cluster-policy`	`- "*.svc"`
	`- "*.svc.cluster.local"`
	`- "*.ocp.fcbint.net"`
		`"*.cluster.local"`
		`"*.ocp.fcbint.net"`

Some of the common attributes across all the policies are the PrivateKey algorithm, min-Max key size, duration, and rotationPolicy.

```
        maxDuration: 8760h0m0s( 1 year)
privateKey:
  algorithm: RSA
  minSize: 2048
  maxSize: 4096
  rotationPolicy: Always
```

Issuers

We support the following `ClusterIssuer` objects

Name	Kind	Group	Notes
keyfactor	ClusterIssuer	command-issuer.keyfactor.com	Requires custom metadata (see example above)

What happens once CertificateRequest is approved

1. Once the certificate request is approved a secret with the name defined in the cert request is created in the namespace with the tls.crt, tls.key and the ca.crt chain. This secret can then be referenced in the deployments or ingress certificates.

2. Once a certificate is created by CM it is then constantly monitored and is auto rotated based on the value given in the duration field for its lifetime.

Approvals and Denial logic explained

- If at least one policy permits the request, the request is approved. If at least one policy is appropriate for the request but none of those permit the request, the request is denied.
- A denied CertificateRequest is considered to be permanently failed. If it was created for a Certificate resource, the issuance will be retried with exponential backoff like all other permanent issuance failures.
- A CertificateRequest that is neither approved nor denied (because no matching policy was found) will not be further processed by cert-manager until it gets either approved or denied.
- A denied request is considered to be permanently failed.
- CertificateRequestPolicies are cluster scoped resources that can be thought of as "policy profiles".
- If Cert-Manager approves the certificate, Keyfactor may still reject it. Please ensure you are using the appropriate metadata as shown in the examples.

Use case Object Reference Examples

Certificate.yaml example

```
       apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  annotations:
    metadata.command-issuer.keyfactor.com/Certificate-Support-Group-
Contact: GroupITDevOps@svb.com
    metadata.command-issuer.keyfactor.com/Environment: Non-Prod-UAT
    metadata.command-issuer.keyfactor.com/Description: "Example
Certificate"
    metadata.command-issuer.keyfactor.com/Organization: FCB
    metadata.command-issuer.keyfactor.com/Application-ID: APM0001255
    metadata.command-issuer.keyfactor.com/Platform: OPENSHIFT
  name: <your-cert-object-name>
  namespace: <your-namespoace-name>
spec:
  commonName: <your service name>.apps.dv-p.ocp.fcbint.net
  dnsNames:
  - <your service name>.apps.dv-p.ocp.fcbint.net
  - <your service name>.apps.dv.ocp.fcbint.net
  - <your service name>.<namespace name>.svc
  - <your service name>.<namespace name>.svc.cluster.local
  duration: 2160h
  issuerRef:
    group: command-issuer.keyfactor.com
    kind: ClusterIssuer
    name: keyfactor
  privateKey:
    algorithm: RSA
    encoding: PKCS1
    size: 4096
  renewBefore: 360h
  secretName: <your service name>-tls
  secretTemplate:
    labels:
      app.kubernetes.io/name:
      app.kubernetes.io/instance:
      app.kubernetes.io/component:
      fcbint.net/app-id: ""
```

Ingress.yaml example

Ingress Object

```
       apiVersion: networking.k8s.io/v1
```

```
kind: Ingress
metadata:
  annotations:
    meta.helm.sh/release-name: example-release
    meta.helm.sh/release-namespace: example-ns-cz
    route.openshift.io/termination: reencrypt ## Recommended
  labels:
  name: <your-service-name>-ingress
  namespace: <your-namespace-name>
spec:
  ingressClassName: openshift-core-zone
  rules:
  - host: <your-service-name>.apps.dv.ocp.fcbint.net
    http:
      paths:
      - backend:
          service:
            name: <your-service-name>
            port:
              name: https
        path: /
        pathType: Prefix
  tls:
  - hosts:
    - <your-service-name>.apps.dv.ocp.fcbint.net
    - <your-service-name>.apps.dv-p.ocp.fcbint.net
    secretName: <your-service-name>-tls
```

With the above objects, you can choose to mount the certificate into your pod to serve the ssl both internally and externally of the cluster, or simply use `edge` termination and leave the certificate to be served by the Ingress object only.

`ingressClassName: should be one of openshift-core-zone or openshift-secure-zone`

Please see the Ingress Process for other considerations.

Upgrade/Patching schedule

1. Cert-manager and its corresponding components will be updated during monthly patching cycle in pre-prod and prod respectively.
2. All upgrades and patches are done via code and reviewed and approved by Container and Cybersec teams
3. If a patch/upgrade fails our first course of action would be to find and fix the issue and resolve it. If resolution does not happen within the change window then the component will be reverted to the older working version.

Validation

Post deployment and patching following are some of the steps for validation

1. Ensure all the pods in the cert-manager namespace are up and running without any issues (like Crashloopback or Imagepull errors)
2. Validate by submitting a dummy cert request to see if CM successfully issues and signs/renews a certificate.

How do I validate the cert I created via cert-manager

- Log into the cluster & switch to the required namespace
- Run the commands
  - `oc get certificates n <namespace name>` this will list the certificate objects created in the namespace.
    - blocked URL
  - `oc get certificaterequest -n <namespace name>` this lists the certificate requests and its status. The Approved column should be true indicating the certificaterequest has been approved for the requested cert object and the cert is ready to be used
    - blocked URL
- You can then use the Openssl command to grep the contents of the route certificate and check for its validity

HowdoIuseSpecificURLCertificatesinmyroutes?

What do i have to do as an app owner to migrate to keyfactor

1. Create a certificate object for all the services/ external facing routes. Refer to - Use case Object Reference Examples section certificate.yaml

2. Create an ingress object for all the routes that you currently have. (Ingress object will create a route object). Refer to - Use case Object Reference Examples section certificate.yaml

    a. The ingress-controller successfully injects the cert-manager certs only via an ingress object. So it is absolutely **mandatory to move away from route objects to ingress objects** incorder to use cert-manager.
    b. there WILL be a small downtime if we are moving from route to ingress managed routes because Ingress object creates the route for you so the existing routes conflicts with new route creation preventing a new route from being crated.
        i. current routes have to be deleted ahead of time before creating ingress objects unless we are using new hostnames entirely.
2. If you are going to use the certificate only for the external routes then its enough to provide the route domain in the dnsNames field.
3. But if we are going to use inter-service/service-service/inter-namespace communication using TLS then its recommended to add the serviceName.Namespace.svc & serviceName. Namespace.svc.local domain also in the dnsNames field.
    a. If its a tomcat service, then the tomcat should know about the certificate issued by cert manager if we are doing re-encrypt.
    b. Openshift platform a.t.m doenst allow passthrough by policy. so only edge and re-encrypt are the only route terminations. refer:

Redis Certificate Limitations

TODO

FYI

Route terminations

A secured route specifies the TLS termination of the route. The available types of termination are listed below:

Edge Termination With edge termination, TLS termination occurs at the router, before the traffic gets routed to the pods. TLS certificates are served by the router, so they must be configured into the route, otherwise the router's default certificate is used for TLS termination. Because TLS is terminated at the router, connections from the router to the endpoints over the internal network are not encrypted.

Pass-through Termination With pass-through termination, encrypted traffic is sent straight to the destination pod without the router providing TLS termination. No key or certificate is required. The destination pod is responsible for serving certificates for the traffic at the endpoint.

Re-encryption Termination Re-encryption is a variation on edge termination, where the router terminates TLS with a certificate, then re-encrypts its connection to the endpoint, which might have a different certificate. Therefore the full path of the connection is encrypted, even over the internal network.

References:

1. `Cert-manager - https://cert-manager.io/docs/`
2. `approver-policy - https://cert-manager.io/docs/projects/approver-policy/`
3. `How to use TLS secrets in your objects -https://medium.com/avmconsulting-blog/how-to-secure-applications-on-kubernetes-ssl-tls-certificates-8f7f5751d788`